Building an End-to-End Data Analytics Pipeline Using Snowflake, Airflow, dbt, and Tableau

Xu Wang, Elina Yin
Department of Data Science/ Data 226
San Jose State University
Email: xu.wang@sjsu.edu, elina.yin@sjsu.edu

***Abstract*—This report presents the design and implementation of an end-to-end data analytics pipeline using Snowflake, Apache Airflow, dbt, and Tableau. Historical weather data is extracted from an external API, loaded into a Snowflake raw layer through Airflow, transformed into analytical tables with dbt, and visualized through an interactive dashboard. The project demonstrates the integration of ETL, ELT, orchestration, testing, snapshotting, and business intelligence reporting in a modern cloud-based analytics workflow.**

***Index Terms*—Snowflake, Airflow, dbt, Tableau, ETL, ELT, Data Warehouse, Analytics Pipeline**

I. Introduction

Modern analytics systems require reliable ingestion, orchestration, transformation, and visualization. This lab focuses on building a complete analytics workflow using Snowflake as the cloud data warehouse, Apache Airflow for orchestration, dbt for ELT transformations, and Tableau for business intelligence reporting. The selected use case is historical weather analytics, where raw weather data is transformed into analytical metrics such as seven-day moving average temperature and cumulative rainfall.

II. Problem Statement

The objective of this lab is to build an end-to-end analytics pipeline that automates raw data ingestion, warehouse loading, transformation, and visualization. The system should load weather data into Snowflake raw tables, execute dbt transformations after ETL completion, and provide a dashboard for interactive exploration of transformed metrics.

III. Requirements and Specifications

The project was implemented to satisfy the following requirements:

- ETL pipeline implemented in Python using Airflow.
- Raw data loaded into Snowflake.
- ELT transformations implemented using dbt.
- dbt workflow scheduled after ETL in Airflow.
- Dashboard visualization built from transformed analytical tables.
- Screenshots of Airflow workflows, dbt commands, and dashboard views.
- GitHub repository containing project code and configuration files.

IV. Dataset Description

The project uses historical weather data collected from the Open-Meteo API. The dataset contains daily observations for selected locations and includes temperature and rainfall related attributes.

A. Source

- Source: Open-Meteo API
- Domain: Historical weather analytics
- Granularity: Daily observations by location

B. Selected Attributes

- `location_name`
- `date`
- `temp_max`
- `temp_min`
- `temp_mean`
- `precipitation`
- `weather_code`

V. System Architecture

Fig. 1 illustrates the overall system architecture.

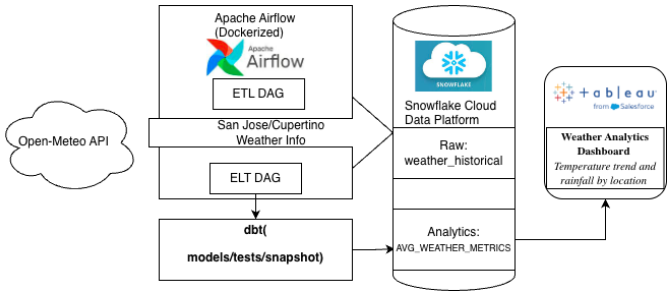


Fig. 1. Overall system architecture.

The pipeline begins by extracting historical weather data from the Open-Meteo API through an Airflow DAG. The processed data is then loaded into Snowflake raw tables. After ETL completion, Airflow triggers a dbt DAG that transforms the raw layer into an analytics layer. Finally, the transformed data is exported for Tableau-based dashboard creation.

VI. Snowflake Data Warehouse Design

A. Raw Layer Table

Table I shows the schema of the raw weather table.

TABLE I
RAW.WEATHER_HISTORICAL Table Structure

Field	Type	Description
location_name	STRING	Name of the location
date	DATE	Observation date
temp_max	FLOAT	Maximum daily temperature
temp_min	FLOAT	Minimum daily temperature
temp_mean	FLOAT	Mean daily temperature
precipitation	FLOAT	Daily rainfall amount
weather_code	INTEGER	Encoded weather condition

B. Analytics Layer Table

Table II shows the transformed analytical table produced by dbt.

TABLE II
ANALYTICS.AVG_WEATHER_METRICS Table Structure

Field	Type	Description
location_name	STRING	Name of the location
date	DATE	Observation date
moving_avg_7d	FLOAT	Seven-day moving average temperature
cumulative_rainfall	FLOAT	Running cumulative rainfall

C. Constraints and Data Quality

Data quality was enforced using dbt tests. The project includes not-null checks and a combined uniqueness test on `location_name` and `date`.

VII. Airflow ETL Pipeline

A. Workflow Overview

The ETL workflow in this lab extends the architecture developed in Lab 1, with an additional step to trigger the dbt-based ELT workflow after the raw data loading process is completed. The workflow consists of the following stages:

1) Extract weather data from the API.
2) Transform the raw response into a structured format.
3) Load records into Snowflake raw tables.
4) Trigger the dbt DAG after ETL completion.

B. Use of Airflow Connections and Variables

Airflow Connections were used to securely configure the Snowflake connection. Airflow Variables were used for runtime configuration such as target table names, API settings, and selected locations.

C. Idempotency and Error Handling

To support reliable re-execution, the ETL workflow used SQL transaction control with rollback support. This design ensures that incomplete or failed loads do not leave partial records in the raw table.

D. Airflow Code Snippet

Listing 1 shows the core structure of the Airflow ETL DAG. The workflow extracts weather data, transforms it into a structured format, loads it into Snowflake, and triggers the dbt DAG after successful ETL completion.

Listing 1. Core Airflow ETL DAG Snippet

```
from airflow import DAG
from airflow.decorators import task
from airflow.operators.trigger_dagrun import
    TriggerDagRunOperator
from datetime import datetime
try:
   conn = hook.get_conn()
   cur = conn.cursor()
   cur.execute("BEGIN;")
   # location_name + date as composite primary key
       to avoid duplicates when loading multiple
       cities
   pass
   cur.execute(f"""␣CREATE␣TABLE␣IF␣NOT␣EXISTS␣{
       target_table}␣)""")
   cur.execute(f"DELETE␣FROM␣{target_table}")
   # DataFrame convert to list of tuples for
       executemany
   pass
   cur.executemany(sql, rows)
   cur.execute("COMMIT;")
   print(f"Successfully␣loaded␣{len(rows)}␣rows␣for␣
       multiple␣locations!")
except Exception as e:
   if conn and cur:
      cur.execute("ROLLBACK;")
      raise e
finally:
   if cur: cur.close()
   if conn: conn.close()
with DAG(
   dag_id="ETL_Weather",
   start_date=datetime(2026, 2, 28),
   schedule="30␣2␣*␣*␣*",
   catchup=False,
   tags=["Lab2", "ETL"]
) as dag:

   @task
   def extract():
      # call Open-Meteo API and return raw response
      pass

   @task
   def transform(raw_data):
      # transform API response into structured rows
      pass

   @task
   def load_V2(records):
      # load records into Snowflake RAW.
          WEATHER_HISTORICAL
      pass

   weather_raw_data = extract()
   transformed_data = transform(weather_raw_data)
   load_task = load_V2(transformed_data)

   trigger_dbt = TriggerDagRunOperator(
      task_id="trigger_dbt_dag",
      trigger_dag_id="Lab2_dbt_Workflow"
   )

   load_task >> trigger_dbt
```

E. Airflow Workflow Screenshots

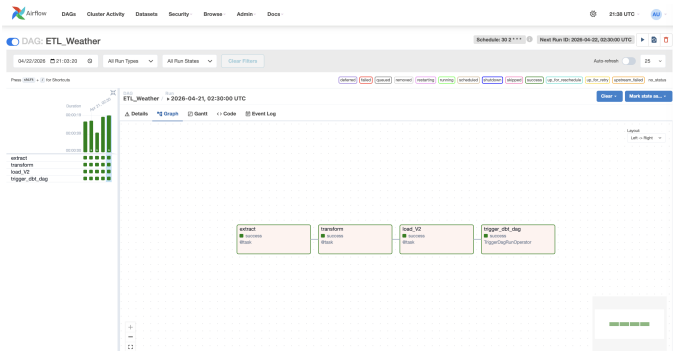


Fig. 2. Airflow ETL DAG execution.

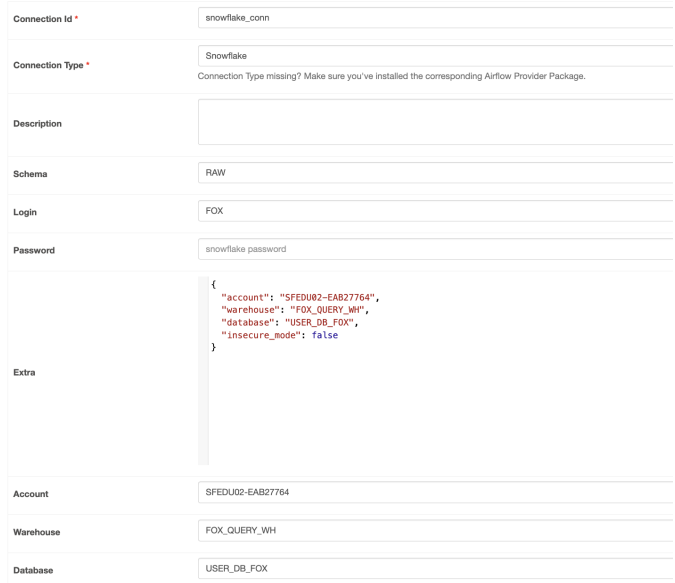


Fig. 3. Airflow connections configuration.

VIII. DBT ELT Workflow

A. dbt Project Design

The dbt project transforms raw weather data into analytics-ready metrics. The implemented transformations include:

- seven-day moving average temperature,
- cumulative rainfall by location,
- data quality tests,
- snapshot generation.

B. dbt Models, Tests, and Snapshots

The dbt project includes:

- model: `avg_weather_metrics.sql`,
- tests: not-null and unique combination tests,
- snapshot: weather historical snapshot.

C. Airflow Scheduling for dbt

A separate Airflow DAG was created for dbt execution. This DAG runs after ETL and executes:

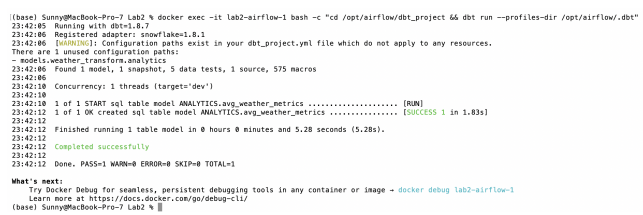
- `dbt run`,
- `dbt test`,
- `dbt snapshot`.

D. dbt Code Snippet

Listing 2. Sample dbt Model Snippet

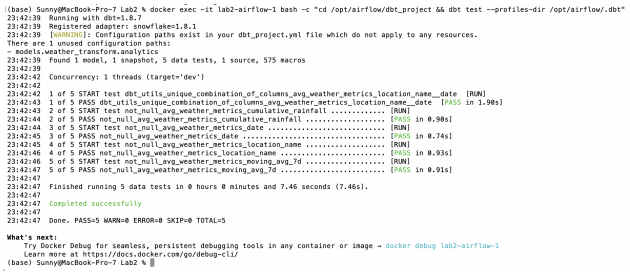
```
with DAG(
   dag_id='Lab2_dbt_Workflow',
   start_date=datetime(2026, 2, 28),
   description='A␣sample␣Airflow␣DAG␣to␣invoke␣dbt␣
       runs␣using␣a␣BashOperator',
   catchup=False,
   schedule=None,
   tags=['Lab2', 'dbt', 'ELT']
) as dag:

   # set dbt project path
   dbt_path = "/opt/airflow/dbt_project"
   profiles_dir = "/opt/airflow/.dbt"

   # 1. dbt run
   dbt_run = BashOperator(
      task_id='dbt_run',
      bash_command=f'cd␣{dbt_path}␣&&␣dbt␣run␣--
          profiles-dir␣{profiles_dir}',
      env=get_dbt_env(),
      append_env=True,
   )

   # 2. dbt test
   dbt_test = BashOperator(
      task_id='dbt_test',
      bash_command=f'cd␣{dbt_path}␣&&␣dbt␣test␣--
          profiles-dir␣{profiles_dir}',
      env=get_dbt_env(),
      append_env=True,
   )

   # 3. dbt snapshot
   dbt_snapshot = BashOperator(
      task_id='dbt_snapshot',
      bash_command=f'cd␣{dbt_path}␣&&␣dbt␣snapshot␣
          --profiles-dir␣{profiles_dir}',
      env=get_dbt_env(),
      append_env=True,
   )

   # define dependencies
   dbt_run >> dbt_test >> dbt_snapshot
```

E. dbt Execution Screenshots



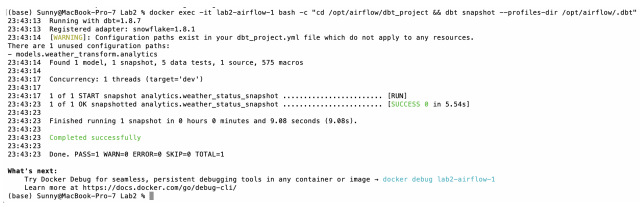
```
(base) Sunny@MacBook-Pro-7 Lab2 % docker exec -it lab2-airflow-1 bash -c "cd /opt/airflow/dbt_project && dbt run --profiles-dir /opt/airflow/.dbt"
23:42:05  Running with dbt=1.8.7
23:42:06  Registered adapter: snowflake=1.8.1
23:42:06  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.weather_transform.analytics
23:42:06  Found 1 model, 1 snapshot, 5 data tests, 1 source, 575 macros
23:42:06
23:42:10  Concurrency: 1 threads (target='dev')
23:42:10
23:42:10  1 of 1 START sql table model ANALYTICS.avg_weather_metrics ..................... [RUN]
23:42:12  1 of 1 OK created sql table model ANALYTICS.avg_weather_metrics ................ [SUCCESS 1 in 1.83s]
23:42:12
23:42:12  Finished running 1 table model in 0 hours 0 minutes and 5.28 seconds (5.28s).
23:42:12
23:42:12  Completed successfully
23:42:12
23:42:12  Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug lab2-airflow-1
    Learn more at https://docs.docker.com/go/debug-cli/
(base) Sunny@MacBook-Pro-7 Lab2 %
```

Fig. 4. dbt run output.



```
(base) Sunny@MacBook-Pro-7 Lab2 % docker exec -it lab2-airflow-1 bash -c "cd /opt/airflow/dbt_project && dbt test --profiles-dir /opt/airflow/.dbt"
23:42:39  Running with dbt=1.8.7
23:42:39  Registered adapter: snowflake=1.8.1
23:42:39  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.weather_transform.analytics
23:42:39  Found 1 model, 1 snapshot, 5 data tests, 1 source, 575 macros
23:42:39
23:42:42  Concurrency: 1 threads (target='dev')
23:42:42
23:42:42  1 of 5 START test dbt_utils_unique_combination_of_columns_avg_weather_metrics_location_name__date  [RUN]
23:42:43  1 of 5 PASS dbt_utils_unique_combination_of_columns_avg_weather_metrics_location_name__date  [PASS in 1.90s]
23:42:43  2 of 5 START test not_null_avg_weather_metrics_cumulative_rainfall ............ [RUN]
23:42:44  2 of 5 PASS not_null_avg_weather_metrics_cumulative_rainfall ................... [PASS in 0.90s]
23:42:44  3 of 5 START test not_null_avg_weather_metrics_date ........................... [RUN]
23:42:45  3 of 5 PASS not_null_avg_weather_metrics_date ................................ [PASS in 0.74s]
23:42:45  4 of 5 START test not_null_avg_weather_metrics_location_name ................... [RUN]
23:42:46  4 of 5 PASS not_null_avg_weather_metrics_location_name ........................ [PASS in 0.93s]
23:42:46  5 of 5 START test not_null_avg_weather_metrics_moving_avg_7d ................... [RUN]
23:42:47  5 of 5 PASS not_null_avg_weather_metrics_moving_avg_7d ........................ [PASS in 0.91s]
23:42:47
23:42:47  Finished running 5 data tests in 0 hours 0 minutes and 7.46 seconds (7.46s).
23:42:47
23:42:47  Completed successfully
23:42:47
23:42:47  Done. PASS=5 WARN=0 ERROR=0 SKIP=0 TOTAL=5

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug lab2-airflow-1
    Learn more at https://docs.docker.com/go/debug-cli/
(base) Sunny@MacBook-Pro-7 Lab2 %
```

Fig. 5. dbt test output.



```
(base) Sunny@MacBook-Pro-7 Lab2 % docker exec -it lab2-airflow-1 bash -c "cd /opt/airflow/dbt_project && dbt snapshot --profiles-dir /opt/airflow/.dbt"
23:43:13  Running with dbt=1.8.7
23:43:13  Registered adapter: snowflake=1.8.1
23:43:14  [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.weather_transform.analytics
23:43:14  Found 1 model, 1 snapshot, 5 data tests, 1 source, 575 macros
23:43:14
23:43:17  Concurrency: 1 threads (target='dev')
23:43:17
23:43:17  1 of 1 START snapshot analytics.weather_status_snapshot ........................ [RUN]
23:43:23  1 of 1 OK snapshotted analytics.weather_status_snapshot ....................... [SUCCESS 0 in 5.54s]
23:43:23
23:43:23  Finished running 1 snapshot in 0 hours 0 minutes and 9.08 seconds (9.08s).
23:43:23
23:43:23  Completed successfully
23:43:23
23:43:23  Done. PASS=1 WARN=0 ERROR=0 SKIP=0 TOTAL=1

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug lab2-airflow-1
    Learn more at https://docs.docker.com/go/debug-cli/
(base) Sunny@MacBook-Pro-7 Lab2 %
```

Fig. 6. dbt snapshot output.

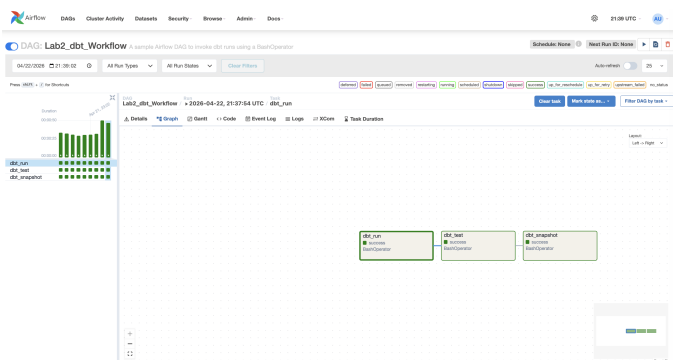


Fig. 7. Airflow dbt DAG execution.

IX. Business Intelligence Dashboard

A. Dashboard Purpose

The dashboard was built to visualize transformed analytical metrics generated by dbt. Its purpose is to allow users to explore weather trends by location and date range.

B. Dashboard Dataset

The dashboard uses the transformed analytics output:

- `USER_DB_FOX.ANALYTICS.AVG_WEATHER_METRICS`

C. Dashboard Components

The dashboard includes:

- location filter,
- date range filter,
- KPI cards,
- moving average temperature trend chart,
- cumulative rainfall trend chart.

D. Dashboard Screenshots

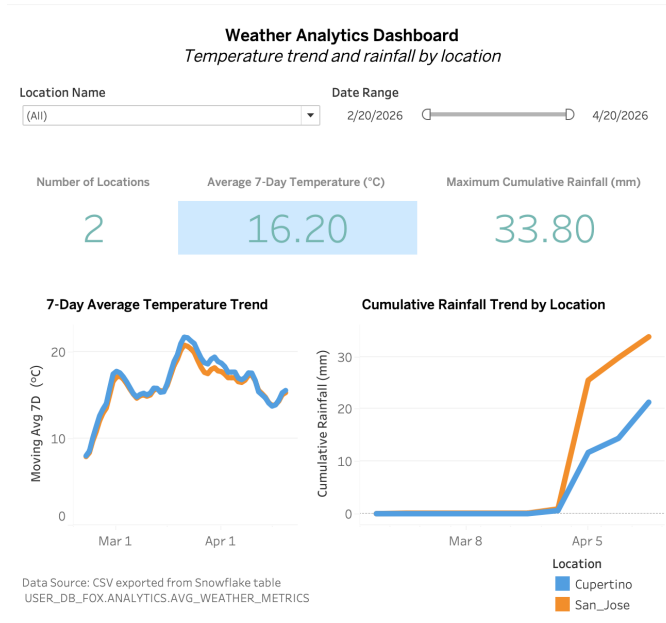


Fig. 8. Weather analytics dashboard.

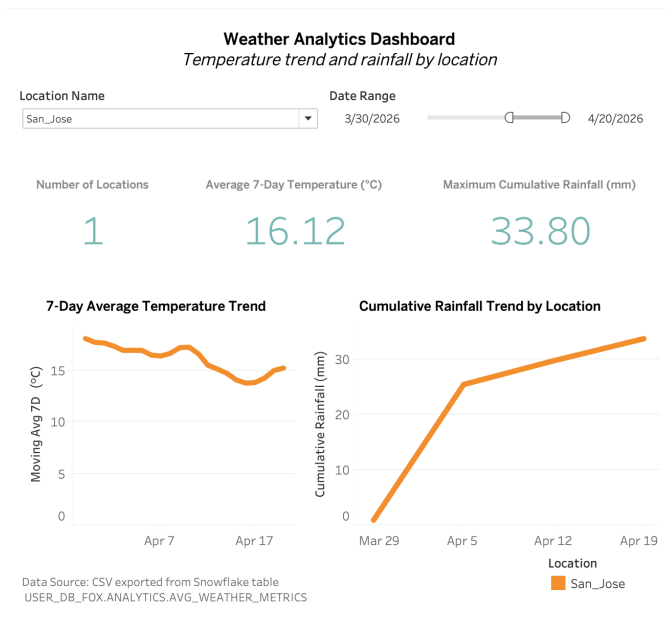


Fig. 9. Dashboard updated with filter selection.

X. GitHub Repository

The GitHub repository contains:

- Airflow DAG files in the `dags/` folder,
- dbt project files in the `weather_transform/` folder,
- Docker and configuration files,
- screenshots and documentation.

Repository link: https://github.com/xuwang-goldenwater/sjsu-data226/tree/main/Lab2

XI. Challenges and Lessons Learned

This lab involved several practical challenges across Docker, Airflow, dbt, Snowflake, and Tableau, which provided valuable lessons in end-to-end analytics pipeline development.

The first major challenge was configuring the Docker Compose environment for Airflow and dbt. During implementation, multiple issues arose from YAML structure, service dependency configuration, and runtime package installation. In particular, relying on `_PIP_ADDITIONAL_REQUIREMENTS` caused long dependency resolution times and unstable container startup behavior. This issue was resolved by moving critical dependencies such as the Snowflake provider and dbt packages into a custom Dockerfile, which made the environment more reproducible and reliable. This highlighted the importance of separating build-time dependency installation from runtime execution.

The second challenge was coordinating Airflow services correctly. Initially, the Airflow webserver became available while the scheduler was not running, which caused DAG execution and refresh issues. In addition, some DAGs failed to load because of import-time configuration errors. Through debugging, it became clear that orchestration systems require not only correct container startup, but also proper service separation and DAG parsing stability.

A third challenge involved Airflow connection management. The dbt DAG depended on a Snowflake connection defined in Airflow, and the DAG initially failed because the required connection ID had not been created. This showed that workflow code should not assume infrastructure configuration is already present. It also demonstrated the importance of using Airflow Connections and Variables in a controlled and explicit way.

The fourth challenge was ensuring consistency across the dbt project. Several errors were caused by mismatched source names, duplicated source definitions, and outdated references inside snapshot files. For example, inconsistent naming between `weather_raw` and `snowflake_raw`, or between `weather_historical` and earlier table names, led to compilation failures. These issues emphasized that dbt projects require strict naming consistency across `sources.yml`, model SQL files, schema files, and snapshots.

Another important challenge was related to environment variables and command execution inside Airflow. Although dbt was successfully installed inside the container, Airflow tasks still failed with `dbt: command not found`. The root cause was that the environment passed to `BashOperator` overwrote the default system environment, which removed the container's executable path. This was resolved by appending the existing environment during task execution. The lesson learned is that orchestration code must preserve the execution environment needed by installed tools.

A further challenge appeared in the dashboarding stage. Tableau Public does not support direct Snowflake connectivity in the same way as Tableau Desktop, which required exporting the transformed Snowflake table as a CSV file before building the dashboard. In addition, some KPI values initially appeared inconsistent with the charts because cumulative rainfall had already been aggregated in dbt and was being re-aggregated incorrectly in Tableau. This highlighted the importance of understanding aggregation context in visualization tools and ensuring that dashboard calculations match warehouse logic.

Overall, these challenges showed that building an end-to-end analytics workflow is not only about implementing individual components, but also about making sure configuration, naming, scheduling, dependency management, and visualization logic are all aligned. The lab provided valuable hands-on experience in debugging real integration issues and improved understanding of how modern data engineering tools work together in practice.

XII. Future Work

Future improvements may include:

- adding more advanced weather indicators,
- supporting incremental loading,
- improving dashboard styling and interaction,
- extending the pipeline for alerting and monitoring.

XIII. Conclusion

This lab successfully implemented an end-to-end analytics pipeline using Snowflake, Airflow, dbt, and Tableau. The workflow automates ingestion, transformation, validation, and reporting for historical weather data. The final solution demonstrates how modern data engineering and analytics tools can be integrated into a functional warehouse-driven reporting system.

References

[1] Apache Airflow Documentation. [Online]. Available: https://airflow.apache.org/

[2] dbt Documentation. [Online]. Available: https://docs.getdbt.com/

[3] Snowflake Documentation. [Online]. Available: https://docs.snowflake.com/

[4] Tableau Documentation. [Online]. Available: https://help.tableau.com/

[5] Tableau Public Dashboard.[Online]. Available:https://public.tableau.com/app/profile/xu.wang6160/viz/SJSU_Data226_Lab2_WeatherTranform/Dashboard1?publish=yes/

[6] Open-Meteo API Documentation. [Online]. Available:https://open-meteo.com/

[7] Github Link. [Online]. Available:https://github.com/xuwang-goldenwater/sjsu-data226/tree/main/Lab2

[8] Docker, "Compose file reference." [Online]. Available: https://docs.docker.com/reference/compose-file/

[9] YAML Language Development Team, "YAML Ain't Markup Language (YAML) Version 1.2.2 Specification." [Online]. Available: https://yaml.org/spec/1.2.2/